13 Persisting data reactively

This chapter covers

- Reactive relational persistence with R2DBC
- Defining reactive repositories for MongoDB and Cassandra
- Testing reactive repositories

If we've learned one thing from science fiction, it's that if you want to improve upon past experiences, all you need is a little time travel. It worked in *Back to the Future*, several episodes of various Star Trek shows, *Avengers: Endgame*, and Stephen King's *11/22/63*. (OK, well maybe that last one didn't turn out better. But you get the idea.)

In this chapter, we're going to rewind back to chapters 3 and 4, revisiting the repositories we created for relational databases, MongoDB, and Cassandra. This time, we're going to improve on them by taking advantage of some of Spring Data's reactive repository support, allowing us to work with those repositories in a nonblocking fashion.

Let's start by looking at Spring Data R2DBC, a reactive alternative to Spring Data JDBC for persistence to relational databases.

13.1 Working with R2DBC

Reactive Relational Database Connectivity, or R2DBC (**https://r2dbc.io/**) as it is commonly known, is a relatively new option for working with relational data using reactive types. It is effectively a reactive alternative to JDBC, enabling nonblocking persistence against conventional relational databases such as MySQL, PostgreSQL, H2, and Oracle. Because it's built on Reactive Streams, it is quite different from JDBC and is a separate specification, unrelated to Java SE.

Spring Data R2DBC is a subproject of Spring Data that offers automatic repository support for R2DBC, much the same as Spring Data JDBC, which we looked at in chapter 3. Unlike Spring Data JDBC, however, Spring Data R2DBC doesn't require strict adherence to domain-driven design concepts. In fact, as you'll soon see, attempting to persist data

through an aggregate root requires a bit more work with Spring Data R2DBC than with Spring Data JDBC.

To use Spring Data R2DBC, you'll need to add a starter dependency to your project's build. For a Maven-built project, the dependency looks like this:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-r2dbc</artifactId>
</dependency>
```

Or, if you're using the Initializr, select the Spring Data R2DBC check box when creating your project.

You'll also need a relational database to persist data to, along with a corresponding R2DBC driver. For our project, we'll be using an in-memory H2 database. Therefore, we need to add two dependencies: the H2 database library itself and the H2 R2DBC driver. The Maven dependencies follow:

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.r2dbc</groupId>
  <artifactId>r2dbc-h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

If you're using a different database, then you'll need to add the corresponding R2BDC driver dependency for the database of your choice.

Now that the dependencies are in place, let's see how Spring Data R2DBC works. Let's start by defining the domain entities.

3.1.1 Defining domain entities for R2DBC

To get to know Spring Data R2DBC, we'll recreate just the persistence layer of the Taco Cloud application, focusing only on the components that are necessary for persisting

taco and order data. This includes creating domain entities for `TacoOrder`, `Taco`, and `Ingredient`, along with corresponding repositories for each.

The first domain entity class we'll create is the `Ingredient` class. It will look something like the next code listing.

Listing 13.1 The `Ingredient` entity class for R2DBC persistence

```
package tacos;

import org.springframework.data.annotation.Id;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.NoArgsConstructor;
import lombok.NonNull;
import lombok.RequiredArgsConstructor;

@Data
@NoArgsConstructor
@RequiredArgsConstructor
@EqualsAndHashCode(exclude = "id")
public class Ingredient {

  @Id
  private Long id;

  private @NonNull String slug;

  private @NonNull String name;
  private @NonNull Type type;

  public enum Type {
    WRAP, PROTEIN, VEGGIES, CHEESE, SAUCE
  }

}
```

As you can see, this isn't much different from other incarnations of the `Ingredient` class that we've created before. Note the following two noteworthy differences:

- Spring Data R2DBC requires that properties have setter methods, so rather than define most properties as `final`, they have to be non-`final`. But to help Lombok create a required arguments constructor, we annotate most of the properties with `@NonNull`. This will cause Lombok and the `@RequiredArgsConstructor` annotation to include those properties in the constructor.
- When saving an object through a Spring Data R2DBC repository, if the object's ID property is non-null, it is treated as an update. In the case of `Ingredient`, the `id` property was previously typed as `String` and specified at creation time. But doing that with Spring Data R2DBC results in an error. So, here we shift that `String` ID to a new property named `slug`, which is just a pseudo-ID for the `Ingredient`, and use a `Long` ID property with a value generated by the database.

The corresponding database table is defined in schema.sql like this:

```
create table Ingredient (
  id identity,
  slug varchar(4) not null,
  name varchar(25) not null,
  type varchar(10) not null
);
```

The `Taco` entity class is also quite similar to its Spring Data JDBC counterpart, as shown in the next code.

Listing 13.2 The `Taco` entity class for R2DBC persistence

```
package tacos;

import java.util.HashSet;
import java.util.Set;
import org.springframework.data.annotation.Id;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.NonNull;
import lombok.RequiredArgsConstructor;

@Data
@NoArgsConstructor
@RequiredArgsConstructor
public class Taco {

  @Id
```

```
  private Long id;

  private @NonNull String name;

  private Set<Long> ingredientIds = new HashSet<>();

  public void addIngredient(Ingredient ingredient) {
    ingredientIds.add(ingredient.getId());
  }

}
```

As with the `Ingredient` class, we have to allow for setter methods on the entity's fields, thus the use of `@NonNull` instead of `final`.

But what's especially interesting here is that instead of having a collection of `Ingredient` objects, `Taco` has a `Set<Long>` referencing the IDs of `Ingredient` objects that are part of this taco. `Set` was chosen over `List` to guarantee uniqueness. But why must we use a `Set<Long>` and not a `Set<Ingredient>` for the ingredient collection?

Unlike other Spring Data projects, Spring Data R2DBC doesn't currently support direct relationships between entities (at least not at this time). As a relatively new project, Spring Data R2DBC is still working through some of the challenges of handling relationships in a nonblocking way. This may change in future versions of Spring Data R2DBC.

Until then, we can't have `Taco` referencing a collection of `Ingredient` and expect persistence to just work. Instead, we have the following options when it comes to dealing with relationships:

- *Define entities with references to the IDs of related objects*. In this case, the corresponding column in the database table must be defined with an array type, if possible. H2 and PostgreSQL are two databases that support array columns, but many others do not. Also, even if the database supports array columns, it may not be possible to define the entries as foreign keys to the referenced table, making it impossible to enforce referential integrity.
- *Define entities and their corresponding tables to match each other perfectly.* For collections, this would mean that the referred object would have a column mapping back to the referring table. For example, the table for `Taco` objects would need to have a column that points back to the `TacoOrder` that the `Taco` is a part of.

- *Serialize referenced entities to JSON and store the JSON in a large* `VARCHAR` *column.* This works especially well if there's no need to query through to the referenced objects. It does, however, have potential limits to how big the JSON-serialized object(s) can be due to limits to the length of the corresponding `VARCHAR` column. Moreover, we won't have any way to leverage the database schema to guarantee referential integrity, because the referenced objects will be stored as a simple string value (which could contain anything).

Although none of these options are ideal, after weighing them, we'll choose the first option for the `Taco` object. The `Taco` class has a `Set<Long>` that references one or more `Ingredient` IDs. This means that the corresponding table must have an array column to store those IDs. For the H2 database, the `Taco` table is defined like this:

```
create table Taco (
  id identity,
  name varchar(50) not null,
  ingredient_ids array
);
```

The `array` type used on the `ingredient_ids` column is specific to H2. For PostgreSQL, that column might be defined as `integer[]`. Consult your chosen database documentation for details on how to define array columns. Note that not all database implementations support array columns, so you may need to choose one of the other options for modeling relationships.

Finally, the `TacoOrder` class, as shown in the next listing, is defined using many of the things we've already employed in defining our domain entities for persistence with Spring Data R2DBC.

Listing 13.3 The `TacoOrder` entity class for R2DBC persistence

```
package tacos;

import java.util.LinkedHashSet;
import java.util.Set;
import org.springframework.data.annotation.Id;
import lombok.Data;

@Data
public class TacoOrder {
```

```
  @Id
  private Long id;

  private String deliveryName;
  private String deliveryStreet;
  private String deliveryCity;
  private String deliveryState;
  private String deliveryZip;
  private String ccNumber;
  private String ccExpiration;
  private String ccCVV;

  private Set<Long> tacoIds = new LinkedHashSet<>();

  private List<Taco> tacos = new ArrayList<>();
  public void addTaco(Taco taco) {
    tacos.add(taco);
  }

}
```

As you can see, aside from having a few more properties, the `TacoOrder` class follows the same pattern as the `Taco` class. It references its child `Taco` objects via a `Set<Long>`. A little later, though, we'll see how to get complete `Taco` objects into a `TacoOrder`, even though Spring Data R2DBC doesn't directly support relationships in that way.

The database schema for the `Taco_Order` table looks like this:

```
create table Taco_Order (
  id identity,
  delivery_name varchar(50) not null,
  delivery_street varchar(50) not null,
  delivery_city varchar(50) not null,
  delivery_state varchar(2) not null,
  delivery_zip varchar(10) not null,
  cc_number varchar(16) not null,
  cc_expiration varchar(5) not null,
  cc_cvv varchar(3) not null,
  taco_ids array
);
```

Just like the `Taco` table, which references ingredients with an array column, the `TacoOrder` table references its child `Taco`s with a `taco_ids` column defined as an array column. Again, this schema is for an H2 database; consult your database documentation for details on support and creation of array columns.

Oftentimes, a production application already has its schema defined through other means, and such scripts aren't desirable except for tests. Therefore, this bean is defined in a configuration that is loaded only when running automated tests and isn't available in the runtime application context. We'll see an example of such a test for testing R2DBC repositories after we have defined those services.

What's more, notice that this bean uses only the schema.sql file from the root of the classpath (under src/main/resources in the project). If you'd like other SQL scripts to be included as part of the database initialization, add more `ResourceDatabasePopulator` objects in the call to `populator.addPopulators()`.

Now that we've defined our entities and their corresponding database schemas, let's create the repositories through which we'll save and fetch taco data.

3.1.2 Defining reactive repositories

In chapters 3 and 4, we defined our repositories as interfaces that extend Spring Data's `CrudRepository` interface. But that base repository interface dealt with singular objects and `Iterable` collections. In contrast, we'd expect that a reactive repository would deal in `Mono` and `Flux` objects.

That's why Spring Data offers `ReactiveCrudRepository` for defining reactive repositories. `ReactiveCrudRepository` operates very much like `CrudRepository`. To create a repository, define an interface that extends `ReactiveCrudRepository`, such as this:

```
package tacos.data;

import org.springframework.data.repository.reactive.ReactiveCrudRepository;

import tacos.TacoOrder;

public interface OrderRepository
    extends ReactiveCrudRepository<TacoOrder, Long> {
}
```

On the surface, the only difference between this `OrderRepository` and the ones we defined in chapters 3 and 4 is that it extends `ReactiveCrudRepository` instead of `CrudRepository`. But what's significantly different is that its methods return `Mono` and `Flux` types instead of a single `TacoOrder` or `Iterable<TacoOrder>`. Two examples include the `findById()` method, which returns a `Mono<TacoOrder>`, and `findAll()`, which returns a `Flux<TacoOrder>`.

To see how this reactive repository might work in action, suppose that you want to fetch all `TacoOrder` objects and print their delivery names to standard output. In that case, you might write some code like the next snippet.

Listing 13.4 Calling a reactive repository method

```
@Autowired
OrderRepository orderRepo;

...

orderRepository.findAll()
   .doOnNext(order -> {
    System.out.println(
       "Deliver to: " + order.getDeliveryName());
   })
   .subscribe();
```

Here, the call to `findAll()` returns a `Flux<TacoOrder>` on which we have added a `doOnNext()` to print the delivery name. Finally, the call to `subscribe()` kicks off the flow of data through the `Flux`.

In the Spring Data JDBC example from chapter 3, `TacoOrder` was the aggregate root, with `Taco` being a child in that aggregate. Therefore, `Taco` objects were persisted as part of a `TacoOrder`, and there was no need to define a repository dedicated to `Taco` persistence. But Spring Data R2DBC doesn't support proper aggregate roots this way, so we'll need a `TacoRepository` through which `Taco` objects are persisted. See the next listing for such a repository.

Listing 13.5 Persisting Taco objects with a reactive repository

```
package tacos.data;

import org.springframework.data.repository.reactive.ReactiveCrudRepository;
```

```
import tacos.Taco;

public interface TacoRepository
      extends ReactiveCrudRepository<Taco, Long> {
}
```

As you can see, `TacoRepository` isn't much different from `OrderRepository`. It extends `ReactiveCrudRepository` to give us reactive types when working with `Taco` persistence. There aren't many surprises here.

On the other hand, `IngredientRepository` is slightly more interesting, as shown next.

Listing 13.6 Persisting `Ingredient` objects with a reactive repository

```
package tacos.data;

import org.springframework.data.repository.reactive.ReactiveCrudRepository;
import reactor.core.publisher.Mono;
import tacos.Ingredient;

public interface IngredientRepository
     extends ReactiveCrudRepository<Ingredient, Long> {

  Mono<Ingredient> findBySlug(String slug);

}
```

As with our other two reactive repositories, `IngredientRepository` extends `ReactiveCrudRepository`. But because we might need a way to look up `Ingredient` objects based on a slug value, `IngredientRepository` includes a `findBySlug()` method that returns a `Mono<Ingredient>`.[1]

Now let's see how to write tests to verify that our repositories work.

13.1.3 Testing R2DBC repositories

Spring Data R2DBC includes support for writing integration tests for R2DBC repositories. Specifically, the `@DataR2dbcTest` annotation, when placed on a test class, causes Spring to create an application context with the generated Spring Data R2DBC repositories as beans that can be injected into the test class. Along with `StepVerifier`, which

we've used in previous chapters, this enables us to write automated tests against all of the repositories we've created.

For the sake of brevity, we'll focus solely on a single test class: `IngredientRepositoryTest`. This will test `IngredientRepository`, verifying that it can save `Ingredient` objects, fetch a single `Ingredient`, and fetch all saved `Ingredient` objects. The next code sample shows this test class.

Listing 13.7 Testing a Spring Data R2DBC repository

```
package tacos.data;

import static org.assertj.core.api.Assertions.assertThat;

import java.util.ArrayList;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.data.r2dbc.DataR2dbcTest;

import reactor.core.publisher.Flux;
import reactor.test.StepVerifier;
import tacos.Ingredient;
import tacos.Ingredient.Type;

@DataR2dbcTest
public class IngredientRepositoryTest {

  @Autowired
  IngredientRepository ingredientRepo;

  @BeforeEach
  public void setup() {
    Flux<Ingredient> deleteAndInsert = ingredientRepo.deleteAll()
        .thenMany(ingredientRepo.saveAll(
            Flux.just(
                new Ingredient("FLTO", "Flour Tortilla", Type.WRAP),
                new Ingredient("GRBF", "Ground Beef", Type.PROTEIN),
                new Ingredient("CHED", "Cheddar Cheese", Type.CHEESE)
        )));

    StepVerifier.create(deleteAndInsert)
            .expectNextCount(3)
            .verifyComplete();
```

```
    }

    @Test
    public void shouldSaveAndFetchIngredients() {

      StepVerifier.create(ingredientRepo.findAll())
        .recordWith(ArrayList::new)
        .thenConsumeWhile(x -> true)
        .consumeRecordedWith(ingredients -> {
          assertThat(ingredients).hasSize(3);
          assertThat(ingredients).contains(
            new Ingredient("FLTO", "Flour Tortilla", Type.WRAP));
          assertThat(ingredients).contains(
            new Ingredient("GRBF", "Ground Beef", Type.PROTEIN));
          assertThat(ingredients).contains(
            new Ingredient("CHED", "Cheddar Cheese", Type.CHEESE));
        })
        .verifyComplete();

      StepVerifier.create(ingredientRepo.findBySlug("FLTO"))
        .assertNext(ingredient -> {
          ingredient.equals(new Ingredient("FLTO", "Flour Tortilla", Type.WRAP));
        });
    }

  }
```

The `setup()` method starts by creating a Flux of test Ingredient objects and then saving them via the `saveAll()` method on the injected `IngredientRepository`. It then uses a `StepVerifier` to verify that, in fact, three ingredients were saved and no more. Internally, the `StepVerifier` subscribes to the ingredient `Flux` to open the flow of data.

In the `shouldSaveAndFetchIngredients()` test method, another `StepVerifier` is used to verify the ingredients returned from the repository's `findAll()` method. It does this by collecting the ingredients into an `ArrayList` via the `recordWith()` method. Then in the lambda passed to `consumeRecordedWith()` method, it simply inspects the contents of the `ArrayList` and verifies that it contains the expected `Ingredient` objects.

At the end of `shouldSaveAndFetchIngredients()`, the `findBySlug()` repository method is tested against a single ingredient by passing `"FLTO"` into `findBySlug()` to create a `Mono>Ingredient<` and then using `StepVerifier` to assert that the next item emitted by the Mono is a flour tortilla `Ingredient` object.

Although we focused only on testing the `IngredientRepository`, the same techniques can be used to test any Spring Data R2BDC–generated repository.

So far, so good. We now have defined our domain types and their respective repositories. And we've written a test to verify that they work. We can use them as is if we like. But these repositories make persistence of a `TacoOrder` inconvenient in that we must first create and persist `Taco` objects that are part of that order and then persist the `TacoOrder` object that references the child `Taco` objects. And when reading the `TacoOrder`, we'll receive only a collection of `Taco` IDs and not fully defined `Taco` objects.

It would be nice if we could persist `TacoOrder` as an aggregate root and have its child `Taco` objects be persisted along with it. Likewise, it would be great if we could fetch a `TacoOrder` and have it fully defined with complete `Taco` objects and not just the IDs. Let's define a service-level class that sits in front of `OrderRepository` and `TacoRepository` to mimic the persistence behavior of chapter 3's `OrderRepository`.

3.1.4 Defining an OrderRepository aggregate root service

The first step toward persisting `TacoOrder` and `Taco` objects together such that `TacoOrder` is the aggregate root is to add a `Taco` collection property to the `TacoOrder` class. This is shown next.

Listing 13.8 Adding a `Taco` collection to `TacoOrder`

```
@Data
public class TacoOrder {

  ...

  @Transient
  private transient List<Taco> tacos = new ArrayList<>();

  public void addTaco(Taco taco) {
    tacos.add(taco);
    if (taco.getId() != null) {
      tacoIds.add(taco.getId());
    }
  }

}
```

Aside from adding a new `List<Taco>` property named `tacos` to the `TacoOrder` class, the `addTaco()` method now adds the given `Taco` to that list (as well as adding its `id` to the `tacoIds` set as before).

Notice, however, that the `tacos` property is annotated with `@Transient` (as well as marked with Java's `transient` keyword). This indicates that Spring Data R2DBC shouldn't attempt to persist this property. Without the `@Transient` annotation, Spring Data R2DBC would try to persist it and result in an error, due to it not supporting such relationships.

When a `TacoOrder` is saved, only the `tacoIds` property will be written to the database, and the `tacos` property will be ignored. Even so, at least now `TacoOrder` has a place to hold `Taco` objects. That will come in handy both for saving `Taco` objects when a `TacoOrder` is saved and also to read in `Taco` objects when a `TacoOrder` is fetched.

Now we can create a service bean that saves and reads `TacoOrder` objects along with their respective `Taco` objects. Let's start with saving a `TacoOrder`. The `TacoOrderAggregateService` class defined in the next code listing has a `save()` method that does precisely that.

Listing 13.9 Saving `TacoOrders` and `Tacos` as an aggregate

```
package tacos.web.api;

import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Service;

import lombok.RequiredArgsConstructor;
import reactor.core.publisher.Mono;
import tacos.Taco;
import tacos.TacoOrder;
import tacos.data.OrderRepository;
import tacos.data.TacoRepository;

@Service
@RequiredArgsConstructor
public class TacoOrderAggregateService {

  private final TacoRepository tacoRepo;
  private final OrderRepository orderRepo;
```

```
  public Mono<TacoOrder> save(TacoOrder tacoOrder) {
    return Mono.just(tacoOrder)
      .flatMap(order -> {
        List<Taco> tacos = order.getTacos();
        order.setTacos(new ArrayList<>());
        return tacoRepo.saveAll(tacos)
            .map(taco -> {
              order.addTaco(taco);
              return order;
            }).last();
      })
      .flatMap(orderRepo::save);
    }

  }
```

Although there aren't many lines in listing 13.9, there's a lot going on in the `save()` method that requires some explanation. Firstly, the `TacoOrder` that is received as a parameter is wrapped in a `Mono` using the `Mono.just()` method. This allows us to work with it as a reactive type throughout the rest of the `save()` method.

The next thing we do is apply a `flatMap()` to the `Mono<TacoOrder>` we just created. Both `map()` and `flatMap()` are options for doing transformations on a data object passing through a `Mono` or `Flux`, but because the operations we perform in the course of doing the transformation will result in a `Mono<TacoOrder>`, the `flatMap()` operation ensures that we continue working with a `Mono<TacoOrder>` after the mapping and not a `Mono<Mono<TacoOrder>>`, as would be the case if we used `map()` instead.

The purpose of the mapping is to ensure that the `TacoOrder` ends up with the IDs of the child `Taco` objects and saves those `Taco` objects along the way. Each `Taco` object's ID is probably `null` initially for a new `TacoOrder`, and we won't know the IDs until after the `Taco` objects have been saved.

After fetching the `List<Taco>` from the `TacoOrder`, which we'll use when saving `Taco` objects, we reset the `tacos` property to an empty list. We'll be rebuilding that list with new `Taco` objects that have been assigned IDs after having been saved.

A call to the `saveAll()` method on the injected `TacoRepository` saves all of our `Taco` objects. The `saveAll()` method returns a `Flux<Taco>` that we then cycle through by way of the `map()` method. In this case, the transformation operation is secondary to the fact that each `Taco` object is being added back to the `TacoOrder`. But to ensure that it's a `TacoOrder` and not a `Taco` that ends up on the resulting `Flux`, the mapping operation

returns the `TacoOrder` instead of the `Taco`. A call to `last()` ensures that we won't have duplicate `TacoOrder` objects (one for each `Taco`) as a result of the mapping operation.

At this point, all `Taco` objects should have been saved and then pushed back into the parent `TacoOrder` object, along with their newly assigned IDs. All that's left is to save the `TacoOrder`, which is what the final `flatMap()` call does. Again, we choose `flatMap()` here to ensure that the `Mono<TacoOrder>` returned from the call to `OrderRepository.save()` doesn't get wrapped in another `Mono`. We want our `save()` method to return a `Mono<TacoOrder>`, not a `Mono<Mono<TacoOrder>>`.

Now let's have a look at a method that will read a `TacoOrder` by its ID, reconstituting all of the child `Taco` objects. The following code sample shows a new `findById()` method for that purpose.

Listing 13.10 Reading `TacoOrders` and `Tacos` as an aggregate

```
public Mono<TacoOrder> findById(Long id) {
   return orderRepo
     .findById(id)
     .flatMap(order -> {
       return tacoRepo.findAllById(order.getTacoIds())
         .map(taco -> {
          order.addTaco(taco);
          return order;
        }).last();
     });
}
```

The new `findById()` method is a bit shorter than the `save()` method. But we still have a lot to unpack in this small method.

The first thing to do is fetch the `TacoOrder` by calling the `findById()` method on the `OrderRepository`. This returns a `Mono<TacoOrder>` that is then flat-mapped to transform it from a `TacoOrder` that has only `Taco` IDs into a `TacoOrder` that includes complete `Taco` objects.

The lambda given to the `flatMap()` method makes a call to the `TacoRepository.findAllById()` method to fetch all `Taco` objects referenced in the `tacoIds` property at once. This results in a `Flux<Taco>` that is cycled over via `map()`, adding each `Taco` to the parent `TacoOrder`, much like we did in the `save()` method after saving all `Taco` objects with `saveAll()`.

Again, the `map()` operation is used more as a means of iterating over the `Taco` objects rather than as a transformation. But the lambda given to `map()` returns the parent `TacoOrder` each time so that we end up with a `Flux<TacoOrder>` instead of a `Flux<Taco>`. The call to `last()` takes the last entry in that `Flux` and returns a `Mono<TacoOrder>`, which is what we return from the `findById()` method.

The code in the `save()` and `findById()` methods may be a little confusing if you're not already in a reactive mind-set. Reactive programming requires a different mindset and can be confusing at first, but you'll come to recognize it as quite elegant as your reactive programming skills get stronger.

As with any code—but especially code that may appear confusing like that in `TacoOrderAggregateService`—it's a good idea to write tests to ensure that it works as expected. The test will also serve as an example of how the `TacoOrderAggregateService` can be used. The following code listing shows a test for `TacoOrderAggregateService`.

Listing 13.11 Testing the `TacoOrderAggregateService`

```
package tacos.web.api;

import static org.assertj.core.api.Assertions.assertThat;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.data.r2dbc.DataR2dbcTest;
import org.springframework.test.annotation.DirtiesContext;

import reactor.test.StepVerifier;
import tacos.Taco;
import tacos.TacoOrder;
import tacos.data.OrderRepository;
import tacos.data.TacoRepository;

@DataR2dbcTest
@DirtiesContext
public class TacoOrderAggregateServiceTests {

  @Autowired
  TacoRepository tacoRepo;

  @Autowired
```

```
OrderRepository orderRepo;

TacoOrderAggregateService service;

@BeforeEach
public void setup() {
  this.service = new TacoOrderAggregateService(tacoRepo, orderRepo);
}

@Test
public void shouldSaveAndFetchOrders() {
  TacoOrder newOrder = new TacoOrder();
  newOrder.setDeliveryName("Test Customer");
  newOrder.setDeliveryStreet("1234 North Street");
  newOrder.setDeliveryCity("Notrees");
  newOrder.setDeliveryState("TX");
  newOrder.setDeliveryZip("79759");
  newOrder.setCcNumber("4111111111111111");
  newOrder.setCcExpiration("12/24");
  newOrder.setCcCVV("123");

  newOrder.addTaco(new Taco("Test Taco One"));
  newOrder.addTaco(new Taco("Test Taco Two"));

  StepVerifier.create(service.save(newOrder))
    .assertNext(this::assertOrder)
    .verifyComplete();

  StepVerifier.create(service.findById(1L))
    .assertNext(this::assertOrder)
    .verifyComplete();
}

private void assertOrder(TacoOrder savedOrder) {
  assertThat(savedOrder.getId()).isEqualTo(1L);
  assertThat(savedOrder.getDeliveryName()).isEqualTo("Test Customer");
  assertThat(savedOrder.getDeliveryName()).isEqualTo("Test Customer");
  assertThat(savedOrder.getDeliveryStreet()).isEqualTo("1234 North Street");
  assertThat(savedOrder.getDeliveryCity()).isEqualTo("Notrees");
  assertThat(savedOrder.getDeliveryState()).isEqualTo("TX");
  assertThat(savedOrder.getDeliveryZip()).isEqualTo("79759");
  assertThat(savedOrder.getCcNumber()).isEqualTo("4111111111111111");
  assertThat(savedOrder.getCcExpiration()).isEqualTo("12/24");
  assertThat(savedOrder.getCcCVV()).isEqualTo("123");
  assertThat(savedOrder.getTacoIds()).hasSize(2);
  assertThat(savedOrder.getTacos().get(0).getId()).isEqualTo(1L);
```

```
    assertThat(savedOrder.getTacos().get(0).getName())
          .isEqualTo("Test Taco One");
    assertThat(savedOrder.getTacos().get(1).getId()).isEqualTo(2L);
    assertThat(savedOrder.getTacos().get(1).getName())
          .isEqualTo("Test Taco Two");
  }

}
```

Listing 13.11 contains a lot of lines, but much of it is asserting the contents of a `TacoOrder` in the `assertOrder()` method. We'll focus on the other parts as we review this test.

The test class is annotated with `@DataR2dbcTest` to have Spring create an application context with all of our repositories as beans. `@DataR2dbcTest` seeks out a configuration class annotated with `@SpringBootConfiguration` to define the Spring application context. In a single-module project, the bootstrap class annotated with `@SpringBootApplication` (which itself is annotated with `@SpringBootConfiguration`) serves this purpose. But in our multimodule project, this test class isn't in the same project as the bootstrap class, so we'll need a simple configuration class like this one:

```
package tacos;

import org.springframework.boot.SpringBootConfiguration;
import org.springframework.boot.autoconfigure.EnableAutoConfiguration;

@SpringBootConfiguration
@EnableAutoConfiguration
public class TestConfig {

}
```

Not only does this satisfy the need for a `@SpringBootConfiguration`-annotated class, but it also enables autoconfiguration, ensuring that (among other things) the repository implementations will be created.

On its own, `TacoOrderAggregateServiceTests` should pass fine. But in an IDE that may share JVMs and Spring application contexts between test runs, running this test alongside other persistence tests may result in conflicting data being written to the in-memory H2 database. The `@DirtiesContext` annotation is used here to ensure that the Spring

application context is reset between test runs, resulting in a new and empty H2 database on each run.

The `setup()` method creates an instance of `TacoOrderAggregateService` using the `TacoRepository` and `OrderRepository` objects injected into the test class. The `TacoOrderAggregateService` is assigned to an instance variable so that the test method(s) can use it.

Now we're finally ready to test our aggregation service. The first several lines of `shouldSaveAndFetchOrders()` builds up a `TacoOrder` object and populates it with a couple of test `Taco` objects. Then the `TacoOrder` is saved via the `save()` method from `TacoOrderAggregateService`, which returns a `Mono<TacoOrder>` representing the saved order. Using `StepVerifier`, we assert that the `TacoOrder` in the returned `Mono` matches our expectations, including that it contains the child `Taco` objects.

Next, we call the service's `findById()` method, which also returns a `Mono<TacoOrder>`. As with the call to `save()`, a `StepVerifier` is used to step through each `TacoOrder` in the returned `Mono` (there should be only one) and asserts that it meets our expectations.

In both `StepVerifier` situations, a call to `verifyComplete()` ensures that there are no more objects in the `Mono` and that the `Mono` is complete.

It's worth noting that although we could apply a similar aggregation operation to ensure that `Taco` objects always contain fully defined `Ingredient` objects, we choose not to, given that `Ingredient` is its own aggregate root, likely being referenced by multiple `Taco` objects. Therefore, every `Taco` will carry only a `Set<Long>` to reference `Ingredient` IDs, which can then be looked up separately via `IngredientRepository`.

Although it may require a bit more work to aggregate entities, Spring Data R2DBC provides a way of working with relational data in a reactive way. But it's not the only reactive persistence option provided by Spring. Let's have a look at how to work with MongoDB using reactive Spring Data repositories.

13.2 Persisting document data reactively with MongoDB

In chapter 4, we used Spring Data MongoDB to define document-based persistence against a MongoDB document database. In this section, we're going to revisit MongoDB persistence using Spring Data's reactive support for MongoDB.

To get started, you'll need to create a project with the Spring Data Reactive MongoDB starter. That is, in fact, the name of the check box to select when creating the project

with the Initalizr. Or you can add it manually to your Maven build with the following dependency:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-mongodb-reactive</artifactId>
</dependency>
```

In chapter 4, we also leaned on the Flapdoodle embedded MongoDB database for testing. Unfortunately, Flapdoodle doesn't behave quite as well when fronted with reactive repositories. When it comes to running the tests, you'll need to have an actual Mongo database running and listening on port 27017.

Now we're ready to start writing code for reactive MongoDB persistence. We'll start with the document types that make up our domain.

3.2.1 Defining domain document types

As before, we'll need to create the classes that define our application's domain. As we do, we'll need to annotate them with Spring Data MongoDB's `@Document` annotation, just as we did in chapter 4, to indicate that they are documents to be stored in MongoDB. Let's start with the `Ingredient` class, shown here.

Listing 13.12 An `Ingredient` class annotated for Mongo persistence

```
package tacos;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
@Document
public class Ingredient {

 @Id
```

```
  private String id;
  private String name;
  private Type type;

  public enum Type {
    WRAP, PROTEIN, VEGGIES, CHEESE, SAUCE
  }

}
```

A keen eye will notice that this `Ingredient` class is identical to the one we created in chapter 4. In fact, MongoDB `@Document` classes are the same whether being persisted through a reactive or nonreactive repository. That means that the `Taco` and `TacoOrder` classes are going to be the same as the ones we created in chapter 4. But for the sake of completeness—and so that you won't need to turn back to chapter 4—we'll repeat them here.

A similarly annotated `Taco` class is shown next.

Listing 13.13 A `Taco` class annotated for Mongo persistence

```
package tacos;

import java.util.ArrayList;
import java.util.Date;
import java.util.List;

import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;
import org.springframework.data.rest.core.annotation.RestResource;

import lombok.Data;

@Data
@RestResource(rel = "tacos", path = "tacos")
@Document
public class Taco {

  @Id
  private String id;
```

```
  @NotNull
  @Size(min = 5, message = "Name must be at least 5 characters long")
  private String name;

  private Date createdAt = new Date();

  @Size(min=1, message="You must choose at least 1 ingredient")
  private List<Ingredient> ingredients = new ArrayList<>();

  public void addIngredient(Ingredient ingredient) {
    this.ingredients.add(ingredient);
  }

}
```

Notice that, unlike `Ingredient`, the `Taco` class isn't annotated with `@Document`. That's because it isn't saved as a document in itself and is instead saved as part of the `TacoOrder` aggregate root. On the other hand, because `TacoOrder` is an aggregate root, it is annotated with `@Document` as shown in the next code.

Listing 13.14 A `TacoOrder` class annotated for Mongo persistence

```
package tacos;

import java.io.Serializable;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import lombok.Data;

@Data
@Document
public class TacoOrder implements Serializable {
  private static final long serialVersionUID = 1L;

  @Id
  private String id;
  private Date placedAt = new Date();

  private User user;
```

```
  private String deliveryName;

  private String deliveryStreet;

  private String deliveryCity;

  private String deliveryState;

  private String deliveryZip;

  private String ccNumber;

  private String ccExpiration;

  private String ccCVV;


  private List<Taco> tacos = new ArrayList<>();

  public void addTaco(Taco taco) {
    tacos.add(taco);
  }

}
```

Again, the domain document classes are no different for reactive MongoDB repositories than they would be for nonreactive repositories. As you'll see next, reactive MongoDB repositories themselves differ very slightly from their nonreactive counterparts.

3.2.2 Defining reactive MongoDB repositories

Now we'll need to define two repositories, one for the `TacoOrder` aggregate root and another for `Ingredient`. We won't need a repository for `Taco` because it is a child of the `TacoOrder` root.

The `IngredientRepository` interface, shown here, should be familiar to you by now:

```
package tacos.data;

import org.springframework.data.repository.reactive.ReactiveCrudRepository;
import org.springframework.web.bind.annotation.CrossOrigin;

import tacos.Ingredient;
```

```
@CrossOrigin(origins="http://localhost:8080")
public interface IngredientRepository
        extends ReactiveCrudRepository<Ingredient, String> {

}
```

This `IngredientRepository` interface is only slightly different from the one we defined in chapter 4 in that it extends `ReactiveCrudRepository` instead of `CrudRepository`. And it differs from the one we created for Spring Data R2DBC persistence only in that it doesn't include the `findBySlug()` method.

Likewise, `OrderRepository` is all but identical to the same MongoDB repository we created in chapter 4, shown next:

```
package tacos.data;

import org.springframework.data.domain.Pageable;
import org.springframework.data.repository.reactive.ReactiveCrudRepository;

import reactor.core.publisher.Flux;
import tacos.TacoOrder;
import tacos.User;

public interface OrderRepository
        extends ReactiveCrudRepository<TacoOrder, String> {

  Flux<TacoOrder> findByUserOrderByPlacedAtDesc(
         User user, Pageable pageable);

}
```

Ultimately, the only difference between reactive and nonreactive MongoDB repositories is whether they extend `ReactiveCrudRepository` or `CrudRepository`. In choosing to extend `ReactiveCrudRepository`, however, clients of these repositories must be prepared to deal with reactive types like `Flux` and `Mono`. That becomes apparent as we write tests for the reactive repositories, which is what we'll do next.

3.2.3 Testing reactive MongoDB repositories

The key to writing tests for MongoDB repositories is to annotate the test class with `@DataMongoTest`. This annotation performs a function similar to the `@DataR2dbcTest` annotation that we used earlier in this chapter. It ensures that a Spring application con-

text is created with the generated repositories available as beans to be injected into the test. From there, the test can use those injected repositories to set up test data and perform other operations against the database.

For example, consider `IngredientRepositoryTest` in the next listing, which tests `IngredientRepository`, asserting that `Ingredient` objects can be written to and read from the database.

Listing 13.15 Testing a reactive Mongo repository

```
package tacos.data;
import static org.assertj.core.api.Assertions.assertThat;
import java.util.ArrayList;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.data.mongo.DataMongoTest;
import reactor.core.publisher.Flux;
import reactor.test.StepVerifier;
import tacos.Ingredient;
import tacos.Ingredient.Type;

@DataMongoTest
public class IngredientRepositoryTest {

 @Autowired
 IngredientRepository ingredientRepo;

 @BeforeEach
 public void setup() {
    Flux<Ingredient> deleteAndInsert = ingredientRepo.deleteAll()
        .thenMany(ingredientRepo.saveAll(
            Flux.just(
                new Ingredient("FLTO", "Flour Tortilla", Type.WRAP),
                new Ingredient("GRBF", "Ground Beef", Type.PROTEIN),
                new Ingredient("CHED", "Cheddar Cheese", Type.CHEESE)
        )));

    StepVerifier.create(deleteAndInsert)
            .expectNextCount(3)
            .verifyComplete();
 }

 @Test
```

```
  public void shouldSaveAndFetchIngredients() {

    StepVerifier.create(ingredientRepo.findAll())
      .recordWith(ArrayList::new)
      .thenConsumeWhile(x -> true)
      .consumeRecordedWith(ingredients -> {
        assertThat(ingredients).hasSize(3);
        assertThat(ingredients).contains(
          new Ingredient("FLTO", "Flour Tortilla", Type.WRAP));
        assertThat(ingredients).contains(
          new Ingredient("GRBF", "Ground Beef", Type.PROTEIN));
        assertThat(ingredients).contains(
          new Ingredient("CHED", "Cheddar Cheese", Type.CHEESE));
      })
      .verifyComplete();

    StepVerifier.create(ingredientRepo.findById("FLTO"))
      .assertNext(ingredient -> {
        ingredient.equals(new Ingredient("FLTO", "Flour Tortilla", Type.WRAP));
      });
  }

}
```

This test is similar to, but still slightly different from, the R2DBC-based repository test we wrote earlier in this chapter. It starts by writing three `Ingredient` objects to the database. Then, it employs two `StepVerifier` instances to verify that `Ingredient` objects can be read through the repository, first as a collection of all `Ingredient` objects and then fetching a single `Ingredient` by its ID.

Also, just as with the R2DBC-based test from earlier, the `@DataMongoTest` annotation will seek out a `@SpringBootConfiguration`-annotated class for creating the application context. A test just like the one created earlier will work here, too.

What's unique here is that the first `StepVerifier` collects all of the `Ingredient` objects into an `ArrayList` and then asserts that the `ArrayList` contains each `Ingredient`. The `findAll()` method doesn't guarantee a consistent ordering of the resulting documents, which makes the use of `assertNext()` or `expectNext()` prone to fail. By collecting all resulting `Ingredient` objects into a list, we can assert that the list has all three objects, regardless of their order.

A test for `OrderRepository` looks quite similar, as shown here.

Listing 13.16 Testing the Mongo OrderRepository

```
package tacos.data;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.data.mongo.DataMongoTest;

import reactor.test.StepVerifier;
import tacos.Ingredient;
import tacos.Taco;
import tacos.TacoOrder;
import tacos.Ingredient.Type;

@DataMongoTest
public class OrderRepositoryTest {

 @Autowired
 OrderRepository orderRepo;

 @BeforeEach
 public void setup() {
  orderRepo.deleteAll().subscribe();
 }
 @Test
 public void shouldSaveAndFetchOrders() {
  TacoOrder order = createOrder();

  StepVerifier
    .create(orderRepo.save(order))
    .expectNext(order)
    .verifyComplete();

  StepVerifier
    .create(orderRepo.findById(order.getId()))
    .expectNext(order)
    .verifyComplete();

  StepVerifier
    .create(orderRepo.findAll())
    .expectNext(order)
    .verifyComplete();
 }

 private TacoOrder createOrder() {
```

```
    TacoOrder order = new TacoOrder();
      ...
    return order;
  }

}
```

The first thing that the `shouldSaveAndFetchOrders()` method does is construct an order, complete with customer and payment information and a couple of tacos. (For brevity's sake, I've left out the details of the `createOrder()` method.) It then uses a `StepVerifier` to save the `TacoOrder` object and assert that the `save()` method returns the saved `TacoOrder`. It then attempts to fetch the order by its ID and asserts that it receives the full `TacoOrder`. Finally, it fetches all `TacoOrder` objects—there should be only one—and asserts it is the expected `TacoOrder`.

As mentioned earlier, you'll need a MongoDB server available and listening on port 27017 to run this test. The Flapdoodle embedded MongoDB doesn't work well with reactive repositories. If you have Docker installed on your machine, you can easily start a MongoDB server exposed on port 27017 like this:

```
$ docker run -p27017:27017 mongo
```

Other ways to get a MongoDB setup are possible. Consult the documentation at **https://www.mongodb.com/** for more details.

Now that we've seen how to create reactive repositories for R2BDC and MongoDB, let's have a look at one more Spring Data option for reactive persistence: Cassandra.

13.3 Reactively persisting data in Cassandra

To get started with reactive persistence against a Cassandra database, you'll need to add the following starter dependency to your project build. This dependency is in lieu of any Mongo or R2DBC dependencies we've used earlier.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-cassandra-reactive</artifactId>
</dependency>
```

Then, you'll need to declare some details about the Cassandra keyspace and how the schema should be managed. In your application.yml file, add the following lines:

```
spring:
  data:
    rest:
      base-path: /data-api
    cassandra:
      keyspace-name: tacocloud
      schema-action: recreate
      local-datacenter: datacenter1
```

This is the same YAML configuration we used in chapter 4 when working with nonreactive Cassandra repositories. The key thing to take note of is the `keyspace-name`. It is important that you create a keyspace with that name in your Cassandra cluster.

You'll also need to have a Cassandra cluster running on your local machine listening on port 9042. The easiest way to do that is with Docker, as follows:

```
$ docker network create cassandra-net
$ docker run --name my-cassandra --network cassandra-net \
      -p 9042:9042 -d cassandra:latest
```

If your Cassandra cluster is on another machine or port, you'll need to specify the contact points and port in application.yml, as shown in chapter 4. To create the keyspace, run the CQL shell and use the `create keyspace` command like this:

```
$ docker run -it --network cassandra-net --rm cassandra cqlsh my-cassandra
cqlsh> create keyspace tacocloud
WITH replication = {'class': 'SimpleStrategy', 'replication_factor' : 1};
```

Now that you have a Cassandra cluster, a new `tacocloud` keyspace, and the Spring Data Cassandra Reactive starter in your project, you're ready to start defining the domain classes.

3.3.1 Defining domain classes for Cassandra persistence

As was the case when persisting with Mongo, the choice of reactive versus nonreactive Cassandra persistence makes absolutely no difference in how you define your domain classes. The domain classes for `Ingredient`, `Taco`, and `TacoOrder` we'll use are identi-

cal to the ones we created in chapter 4. A Cassandra-annotated `Ingredient` class is shown here.

Listing 13.17 Annotating `Ingredient` for Cassandra persistence

```
package tacos;

import org.springframework.data.cassandra.core.mapping.PrimaryKey;
import org.springframework.data.cassandra.core.mapping.Table;

import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
@Table("ingredients")
public class Ingredient {

  @PrimaryKey
  private String id;
  private String name;
  private Type type;

  public enum Type {
    WRAP, PROTEIN, VEGGIES, CHEESE, SAUCE
  }

}
```

As for the `Taco` class, it is defined with similar Cassandra persistence annotations in the next code listing.

Listing 13.18 Annotating `Taco` for Cassandra persistence

```
package tacos;

import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.UUID;
```

```
import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;

import org.springframework.data.cassandra.core.cql.Ordering;
import org.springframework.data.cassandra.core.cql.PrimaryKeyType;
import org.springframework.data.cassandra.core.mapping.Column;
import org.springframework.data.cassandra.core.mapping.PrimaryKeyColumn;
import org.springframework.data.cassandra.core.mapping.Table;
import org.springframework.data.rest.core.annotation.RestResource;

import com.datastax.oss.driver.api.core.uuid.Uuids;

import lombok.Data;

@Data
@RestResource(rel = "tacos", path = "tacos")
@Table("tacos")
public class Taco {

  @PrimaryKeyColumn(type=PrimaryKeyType.PARTITIONED)
  private UUID id = Uuids.timeBased();

  @NotNull
  @Size(min = 5, message = "Name must be at least 5 characters long")
  private String name;

  @PrimaryKeyColumn(type=PrimaryKeyType.CLUSTERED,
                    ordering=Ordering.DESCENDING)
  private Date createdAt = new Date();

  @Size(min=1, message="You must choose at least 1 ingredient")
  @Column("ingredients")
  private List<IngredientUDT> ingredients = new ArrayList<>();

  public void addIngredient(Ingredient ingredient) {
     this.ingredients.add(new IngredientUDT(ingredient.getName(), ingredient.getType()));
  }

}
```

Because `Taco` refers to `Ingredient` objects via a user-defined type, you'll also need the `IngredientUDT` class, as shown next.

Listing 13.19 An `Ingredient` user-defined type for Cassandra persistence

```
package tacos;

import org.springframework.data.cassandra.core.mapping.UserDefinedType;

import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor(access = AccessLevel.PRIVATE, force = true)
@UserDefinedType("ingredient")
public class IngredientUDT {
  private String name;
  private Ingredient.Type type;
}
```

The final of our three domain classes, `TacoOrder` is annotated for Cassandra persistence as shown in the following listing.

Listing 13.20 Annotating `TacoOrder` for Cassandra persistence

```
package tacos;

import java.io.Serializable;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.UUID;

import org.springframework.data.cassandra.core.mapping.Column;
import org.springframework.data.cassandra.core.mapping.PrimaryKey;
import org.springframework.data.cassandra.core.mapping.Table;

import com.datastax.oss.driver.api.core.uuid.Uuids;

import lombok.Data;

@Data
@Table("tacoorders")
public class TacoOrder implements Serializable {
  private static final long serialVersionUID = 1L;
```

```
  @PrimaryKey
  private UUID id = Uuids.timeBased();
  private Date placedAt = new Date();

  @Column("user")
  private UserUDT user;

  private String deliveryName;

  private String deliveryStreet;

  private String deliveryCity;

  private String deliveryState;

  private String deliveryZip;

  private String ccNumber;

  private String ccExpiration;

  private String ccCVV;
  @Column("tacos")
  private List<TacoUDT> tacos = new ArrayList<>();

  public void addTaco(Taco taco) {
    addTaco(new TacoUDT(taco.getName(), taco.getIngredients()));
  }

  public void addTaco(TacoUDT tacoUDT) {
    tacos.add(tacoUDT);
  }

}
```

And, just like how `Taco` refers to `Ingredient` via a user-defined type, `TacoOrder` refers to `Taco` via the `TacoUDT` class, which is shown next.

Listing 13.21 An `Taco` user-defined type for Cassandra persistence

```
package tacos;

import java.util.List;
```

```
import org.springframework.data.cassandra.core.mapping.UserDefinedType;

import lombok.Data;

@Data
@UserDefinedType("taco")
public class TacoUDT {

  private final String name;
  private final List<IngredientUDT> ingredients;

}
```

It bears repeating that these are identical to their nonreactive counterparts. I've only repeated them here so that you don't have to flip back 11 chapters to remember what they look like.

Now let's define the repositories that persist these objects.

3.3.2 Creating reactive Cassandra repositories

By now you may already be expecting the reactive Cassandra repositories to look a lot like the equivalent nonreactive repositories. If so, then great! You're catching on that Spring Data, wherever possible, attempts to maintain a similar programming model regardless of whether or not repositories are reactive.

You may have already guessed that the only key difference that makes the repositories reactive is that the interfaces extend `ReactiveCrudRepository`, as shown here in the `IngredientRepository` interface:

```
package tacos.data;

import org.springframework.data.repository.reactive.ReactiveCrudRepository;

import tacos.Ingredient;

public interface IngredientRepository
      extends ReactiveCrudRepository<Ingredient, String> {

}
```

Naturally, the same holds true for `OrderRepository`, as shown next:

```
package tacos.data;

import java.util.UUID;

import org.springframework.data.domain.Pageable;
import org.springframework.data.repository.reactive.ReactiveCrudRepository;

import reactor.core.publisher.Flux;
import tacos.TacoOrder;
import tacos.User;

public interface OrderRepository
        extends ReactiveCrudRepository<TacoOrder, UUID> {

  Flux<TacoOrder> findByUserOrderByPlacedAtDesc(
          User user, Pageable pageable);

}
```

In fact, not only are these repositories reminiscent of their nonreactive counterparts, they also do not differ greatly from the MongoDB repositories we wrote earlier in this chapter. Aside from Cassandra using `UUID` as an ID type instead of `String` for `TacoOrder`, they are virtually identical. This once again demonstrates the consistency employed (where possible) across Spring Data projects.

Let's wrap up our look at writing reactive Cassandra repositories by writing a couple of tests to verify that they work.

3.3.3 Testing reactive Cassandra repositories

At this point, it may not come as a surprise that testing reactive Cassandra repositories is quite similar to how you test reactive MongoDB repositories. For example, take a look at `IngredientRepositoryTest` in the next listing, and see if you can spot how it differs from listing 13.15.

Listing 13.22 Testing the Cassandra `IngredientRepository`

```
package tacos.data;

import static org.assertj.core.api.Assertions.assertThat;
import java.util.ArrayList;
```

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.data.cassandra
.DataCassandraTest;

import reactor.core.publisher.Flux;
import reactor.test.StepVerifier;
import tacos.Ingredient;
import tacos.Ingredient.Type;

@DataCassandraTest
public class IngredientRepositoryTest {

  @Autowired
  IngredientRepository ingredientRepo;

  @BeforeEach
  public void setup() {
    Flux<Ingredient> deleteAndInsert = ingredientRepo.deleteAll()
        .thenMany(ingredientRepo.saveAll(
            Flux.just(
                new Ingredient("FLTO", "Flour Tortilla", Type.WRAP),
                new Ingredient("GRBF", "Ground Beef", Type.PROTEIN),
                new Ingredient("CHED", "Cheddar Cheese", Type.CHEESE)
        )));

    StepVerifier.create(deleteAndInsert)
            .expectNextCount(3)
            .verifyComplete();
  }

  @Test
  public void shouldSaveAndFetchIngredients() {

    StepVerifier.create(ingredientRepo.findAll())
        .recordWith(ArrayList::new)
        .thenConsumeWhile(x -> true)
        .consumeRecordedWith(ingredients -> {
          assertThat(ingredients).hasSize(3);
          assertThat(ingredients).contains(
              new Ingredient("FLTO", "Flour Tortilla", Type.WRAP));
          assertThat(ingredients).contains(
              new Ingredient("GRBF", "Ground Beef", Type.PROTEIN));
          assertThat(ingredients).contains(
              new Ingredient("CHED", "Cheddar Cheese", Type.CHEESE));
```

```
        })
        .verifyComplete();

    StepVerifier.create(ingredientRepo.findById("FLTO"))
        .assertNext(ingredient -> {
          ingredient.equals(new Ingredient("FLTO", "Flour Tortilla", Type.WRAP));
        });
  }

}
```

Did you see it? Where the MongoDB version was annotated with `@DataMongoTest`, this new Cassandra version is annotated with `@DataCassandraTest`. That's it! Otherwise, the tests are identical.

The same is true for `OrderRepositoryTest`. Replace `@DataMongoTest` with `@DataCassandraTest`, and everything else is the same, as shown here:

```
@DataCassandraTest
public class OrderRepositoryTest {
  ...
}
```

Once again, consistency between various Spring Data projects extends even into how the tests are written. This makes it easy to switch between projects that persist to different kinds of databases without having to think much differently about how they are developed.

Summary

- Spring Data supports reactive persistence for a variety of database types, including relational databases (with R2DBC), MongoDB, and Cassandra.
- Spring Data R2DBC offers a reactive option for relational persistence but doesn't yet directly support relationships in domain classes.
- For lack of direct relationship support, Spring Data R2DBC repositories require a different approach to domain and database table design.
- Spring Data MongoDB and Spring Data Cassandra offer a near-identical programming model for writing reactive repositories for MongoDB and Cassandra databases.
- Using Spring Data test annotations along with `StepVerifier`, you can test automatically created reactive repositories from the Spring application context.

---

[1] This method wasn't necessary in chapter 3's JDBC-based repository because we were able to have the `id` field serve double duty as both an ID and a slug.

Previous chapter

< 12 Developing reactive APIs

Next chapter

14 Working with RSocket >